



FINAL EXERCISE UNIT 2

Library Turbo

SERVICES AND PROCESSES PROGRAMMING

IES SAN VICENTE

1	INTRODUCTION AND FIRST STEPS	2
1.1	SETTING UP THE PROJECT.....	2
1.2	CLASS STRUCTURE	2
1.2.1	<i>HashSHA256 class</i>	2
1.2.2	<i>TextAnalyzer class</i>	2
1.2.3	<i>ReportWriter class</i>	3
1.2.4	<i>ReportVerifier class</i>	3
2	HOW SHOULD IT WORK?.....	4
2.1	PROCESSING MODE	4
2.2	VERIFICATION MODE	4
2.2.1	<i>Verification log files</i>	5
3	JSON OUTPUT EXAMPLE.....	5
4	HELPFUL TIPS	5
5	EVALUATION RULES	8

1 INTRODUCTION AND FIRST STEPS

In this final exercise, we are going to implement a **console Java application** that processes text files concurrently using threads and executors.

This project will help you practice **real concurrent programming**, performing both **I/O** and **CPU** tasks.

While the files are being processed, the program will show the **progress in real time**, printing how many files have been completed every second.

After processing, the program can also **verify** the generated reports, ensuring that no file has been modified.

1.1 SETTING UP THE PROJECT

You need to create a new **Java project** called **LibraryTurbo**.

The source code folder will be divided into these **packages**:

```
src/
├── Main.java                # Entry point: processing & verification
├── analysis/
│   ├── HashSHA256.java      # Computes SHA-256 for files
│   └── TextAnalyzer.java    # Two Streams: words & target
├── matches
│   └── ReportVerifier.java   # Verification mode
├── output/
│   └── ReportWriter.java    # Writes JSON reports and
errors.log
```

1.2 CLASS STRUCTURE

Besides the Main class that handles input and coordination, you must create the following classes.

1.2.1 HASHSHA256 CLASS

This class will contain a static method that calculates **SHA-256** hash of a given file (this method will be provided):

```
public static String sha256(Path filePath) throws IOException,
NoSuchAlgorithmException
```

This hash will be used to identify files and verify later if they have been modified.

1.2.2 TEXTANALYZER CLASS

This class will read text files and analyze them using **two different Streams**:

1. The first stream will count **the total number of words**.
2. The second stream will count **how many times a given target word appears**.

Each stream reads the file line by line, splits the text with a regular expression, and filters empty words.

You must use **two independent Streams**, because a Stream cannot be reused.

1.2.3 REPORTWRITER CLASS

This class will write a **JSON report per file** in a sibling folder called :

```
_report + <originalFolderName>
```

If this folder doesn't exist, it must be automatically created.

Each JSON contains:

- **"path"**: The full path to the file (always using forward slashes /),
- **"processedAt"**: The processing date and time ,
- **"totalWords"**: The total number of words,
- **"targetCount"**: The number of occurrences of the target word,
- **"sha256"**: The SHA-256 of the file.

Each JSON file name will be composed of the first 16 characters of the SHA, followed by the original file name:

```
<sha_prefix>_<filename>.json
```

Example:

```
5a1f3b4e6c8d9a7f_poem.txt.json
```

Errors must be appended to errors.log

1.2.4 REPORTVERIFIER CLASS

This class will implement the verification mode.

It checks all JSON files in:

```
_report<directoryName>
```

Verification steps:

1. Create one **Callable** per JSON file.
2. Launch all callables with **invokeAll**.
3. Extract **path** and **sha256** using **two separate Streams** reading the JSON.
4. Recompute SHA.256 of the real file.
5. If the SHA does not match, write a line to:

```
inconsistencies.txt
```

using a **synchronized block**.

6. Append detailed verification data to:

```
verification.log
```

7. Gather results using:

```
futures.stream().allMatch(...)
```

JSON parsing must be done **manually**, using `split()` and `substring()` methods, since you already know the structure of your own JSON

2 HOW SHOULD IT WORK?

Before starting, you will receive sample text files.

When you start the program, it will:

1. Ask for the directory to process (repeat until valid)
2. Ask the user to choose:
 - **Generate new reports**, or
 - **Verify existing reports**.
3. If generating reports, ask for the word to be count. (target word)

2.1 PROCESSING MODE

- Recursively scan for `.txt` and `.md` files..
- It will use a **ThreadPoolExecutor** to process them concurrently.
- Use a **ScheduledExecutorService** to print this every second:

```
Progress: <completed> / <total> | active=<active> | queued=<queued>
```

- Each file will be processed by a **Runnable**:
 1. Compute the SHA-256.
 2. Analyze the text (two Streams).
 3. Write the JSON report.

2.2 VERIFICATION MODE

1. List all `.json` files inside `_report<directoryName>`
2. For each JSON, create a **Callable<Boolean>**:
 1. Extract "path" and "sha256" using two different streams.
 2. Recompute SHA-256 of the real file.
 3. Compare SHA:
 - If mismatch or missing file → write to `inconsistencies.txt`.
 - All operations, even correct ones, must be written in `verification.log`.
4. Combine results using:

```
invokeAll + futures.stream().allMatch(...)
```

3. If not all JSONs are valid print this:

Not all reports are consistent. See `_report<directoryName>/inconsistencies.txt`

2.2.1 VERIFICATION LOG FILES

Two log files must be generated:

verification.log

Contains detailed information for **every step** of verification.

Typical entries:

```
Checking JSON: <jsonFile> at <timestamp>
Extracted path: <path> and sha256: <sha>
Recomputing SHA for: <path>
SHA result for <path> json=<shaJson> actual=<shaNow>
Mismatch detected for: <path>
Missing file: <path>
Malformed JSON (path/sha missing): <jsonFile>
Error verifying <jsonFile>: <errorMessage>
Future failed: <errorMessage>
Verification finished: OK
Verification finished: NOT OK
```

inconsistencies.txt

Contains **only incorrect cases**:

```
Malformed JSON (path/sha missing): <jsonFile>
Missing file: <file> (from <jsonFile>)
SHA mismatch: <file> (json=<sha>, actual=<shaNow>)
Error verifying <jsonFile>: <message>
Future failed: <message>
```

3 JSON OUTPUT EXAMPLE

```
{
  "path": "C:/Docs/texts/poem.txt",
  "processedAt": "2025-11-12T22:15:48.623+01:00",
  "totalWords": 8,
  "targetCount": 2,
  "sha256": "5a1f3b4e6c8d9a7f3f0b2c1e5d6a8f9e..."
}
```

4 HELPFUL TIPS

- Remember: each Stream can only be consumed once → you need **two separate ones**.
- Use manual `split()` and `substring()` to parse JSON.
- Handle exceptions gracefully and continue processing other files.

5 EXAMPLE OF PROGRAM EXECUTION

5.1 REPORT GENERATION MODE

```
=== Library Turbo ===
Enter the directory to scan: C:/Docs/TextSamples
(G)enerate report or (V)erify report?: g
Enter the target word to count (required): magic

Text files to process: 5

Progress: 0 / 5 done | active=4, queued=1, remaining=5
Progress: 3 / 5 done | active=2, queued=0, remaining=2
Progress: 5 / 5 done | active=0, queued=0, remaining=0

Done. JSON reports written into C:/Docs/_reportTextSamples
```

5.1.1 GENERATED FILES (EXAMPLE)

```
C:/Docs/_reportTextSamples/
├── 12ab9e34cdff1122_story.txt.json
├── 34aa11bb8922ccdd_notes.md.json
├── 98ffa033bb993211_poem.txt.json
└── errors.log
```

5.1.2 VERIFICATION MODE (ALL REPORTS CORRECT)

```
=== Library Turbo ===
Enter the directory to scan: C:/Docs/TextSamples
(G)enerate report or (V)erify report?: v

JSON reports to verify: 5
Verification finished: OK
All reports are consistent.
```

verification.log (partial)

```
Checking JSON:
C:/Docs/_reportTextSamples/12ab9e34cdff1122_story.txt.json at 2025-11-
12T21:30:22.501+01:00
Extracted path: C:/Docs/TextSamples/story.txt and sha256:
12ab9e34cdff1122...
Recomputing SHA for: C:/Docs/TextSamples/story.txt
SHA result for C:/Docs/TextSamples/story.txt json=12ab9e34cdff1122...
actual=12ab9e34cdff1122...
Checking JSON:
C:/Docs/_reportTextSamples/34aa11bb8922ccdd_notes.md.json at 2025-11-
12T21:30:22.610+01:00
Extracted path: C:/Docs/TextSamples/notes.md and sha256:
34aa11bb8922ccdd...
Recomputing SHA for: C:/Docs/TextSamples/notes.md
SHA result: json=34aa11bb8922ccdd... actual=34aa11bb8922ccdd...
Verification finished: OK
```

5.1.3 VERIFICATION MODE (WITH INCONSISTENCIES)

```
=== Library Turbo ===
```

```
Enter the directory to scan: C:/Docs/TextSamples
(G)enerate report or (V)erify report?: v
```

```
JSON reports to verify: 5
Not all reports are consistent. See
C:/Docs/_reportTextSamples/inconsistencies.txt
```

Inconsistencies.txt

```
Missing file: C:/Docs/TextSamples/poem.txt (from poem.txt.json)
SHA mismatch: C:/Docs/TextSamples/story.txt (json=12ab9e34...,
actual=45cc90ee...)
Malformed JSON (path/sha missing): broken.json
```

verification.log (partial)

```
Checking JSON: C:/Docs/_reportTextSamples/poem.txt.json at 2025-11-
12T21:42:10.112+01:00
Extracted path: C:/Docs/TextSamples/poem.txt and sha256: 98ffa033...
Missing file: C:/Docs/TextSamples/poem.txt (from
C:/Docs/_reportTextSamples/poem.txt.json)

Checking JSON: C:/Docs/_reportTextSamples/story.txt.json at 2025-11-
12T21:42:10.251+01:00
Extracted path: C:/Docs/TextSamples/story.txt and sha256: 12ab9e34...
Recomputing SHA for: C:/Docs/TextSamples/story.txt
SHA result: json=12ab9e34... actual=45cc90ee...
Mismatch detected for: C:/Docs/TextSamples/story.txt

Verification finished: NOT OK
```


6 EVALUATION RULES

To get your final mark, the following rules will be applied, and you have to use streams and lambda expressions wherever you can.

Item achieved	Points
Appropriate package and class structure.	0,5
Correct use of <code>ExecutorService</code> with <code>Runnable</code> (processing)	1,5
Live progress using <code>ScheduledExecutorService</code> + <code>ThreadPoolExecutor</code> metrics	1
Text analysis with two Streams (total & target)	2
SHA-256 computed and used in JSON file name	1
JSON content correct + forward slashes (/) in path	0,5
Verification mode: <code>Callable<Boolean></code> + <code>invokeAll</code> + <code>allMatch</code>	1
Synchronized concurrent writes to <code>inconsistencies.txt</code> and <code>verification.log</code>	0,5
Graceful shutdown and error logging	1